UNIVERSIDAD DON BOSCO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA EN COMPUTACIÓN
Desarrollo de Aplic. Web Soft. Interpret. en el Cliente DAW404

Rubrica para trabajo de prueba práctica #03

Indicaciones Generales:

Desarrollar una aplicación web utilizando React, enfocada en la creación y reutilización de componentes para mostrar información de diferentes países y consultar el clima actual de sus capitales. La aplicación deberá consumir datos desde la API pública Rest Countries para mostrar una lista de países mediante componentes reutilizables, donde cada país muestre su nombre y bandera. Al seleccionar un país, la información detallada deberá mostrarse utilizando componentes independientes y organizados, incluyendo el nombre del país y su capital. Además, se deberá consumir la API de OpenWeatherMap para mostrar información climática actual de la capital, incluyendo temperatura, estado del clima, humedad y velocidad del viento. La aplicación deberá implementar buenas prácticas en la estructuración de componentes, manejo de props y estados, validación de datos obtenidos desde las APIs, mensajes de carga mientras se obtiene la información y mensajes de error cuando las APIs no respondan o los datos no sean válidos.

Indicaciones importantes para la entrega

- Entregar el proyecto en repositorio de git (uno por pareja)
- Subir al aula virtual.
- Presentación en clase (semana 17) donde el grupo muestre el funcionamiento de la página.
- Ambos integrantes deben poder explicar cualquier parte del código.

NOTA: Los alumnos deben de cumplir con el 75% de la asistencia a los laboratorios.

RUBRICA DE EVALUACIÓN

1. ARQUITECTURA DE COMPONENTES

1.1 Componentización y reutilización	**15%**	Componentes bien divididos siguiendo el principio de responsabilidad única. Crea componentes reutilizables (CountryCard, WeatherInfo, etc.). No hay duplicación de código. Usa composición efectivamente.	Componentes separados y reutilizables, con pequeñas oportunidades de mejora en la división de responsabilidades o composición.	Algunos componentes son reutilizables pero hay duplicación de código o componentes que hacen demasiado. Falta división clara de responsabilidades.	Todo en un solo componente o componentes excesivamente acoplados. No hay reutilización. Código duplicado abundante.
1.2 Estructura de carpetas y archivos	**5%**	Estructura clara y organizada: /components, /services o /api, /utils, etc. Nombres descriptivos. Convenciones consistentes. Fácil de navegar.	Estructura organizada básica. Componentes en carpeta separada. Convenciones mayormente consistentes.	Estructura mínima. Algunos archivos organizados pero sin criterio claro. Mezcla de convenciones.	Sin estructura. Todos los archivos en la raíz o carpetas sin sentido. Nombres inconsistentes o confusos.

2. MANEJO DE PROPS Y STATE

2.1 Uso correcto de Props	**10%**	Props bien definidos y documentados. Destructuring apropiado. PropTypes o TypeScript para validación. Naming semántico. Patrón de paso de props limpio.	Props usados correctamente. Puede faltar validación (PropTypes) o documentación, pero la comunicación entre componentes funciona bien.	Props usados pero con inconsistencias: props innecesarios, prop drilling excesivo, o naming confuso.	Props mal utilizados o ausentes. Componentes acceden a datos de forma inapropiada. No hay comunicación clara entre componentes.
2.2 Manejo de Estado (useState, useEffect)	**15%**	Estado local manejado con useState de forma apropiada. useEffect implementado correctamente (dependencias, cleanup). Estado elevado cuando es necesario. No hay estados redundantes.	Estado manejado correctamente pero con pequeñas oportunidades de optimización en useEffect o ubicación del estado.	Estado funcional pero con problemas: dependencias faltantes en useEffect, warnings en consola, o estado ubicado incorrectamente.	Estado mal manejado: loops infinitos, useEffect sin dependencias apropiadas, o estado global cuando debería ser local.

3. CONSUMO DE APIs

3.1 API Rest Countries	**10%**	Consume la API correctamente. Implementa servicio/función reutilizable para las llamadas. Maneja la respuesta apropiadamente. Código limpio y mantenible.	Consume la API correctamente. La implementación funciona bien pero podría estar mejor organizada (ej: lógica en el componente en vez de servicio).	Consume la API pero con implementación problemática: múltiples llamadas innecesarias, código duplicado, o warnings.	No consume la API, usa datos hardcodeados, o la implementación falla sin manejo apropiado.
3.2 API OpenWeatherMap	**10%**	Consume la API del clima correctamente. Muestra los 4 datos requeridos (temperatura, estado, humedad, viento). API key en variables de entorno (.env). Implementación limpia.	Consume la API y muestra los datos requeridos. Funciona bien pero la API key puede estar expuesta o faltar algún dato menor.	Consume la API pero muestra solo datos parciales (2-3 de 4) o la implementación tiene problemas de organización.	No consume la API del clima, datos inventados, o la integración falla completamente.

4. INTERFAZ Y EXPERIENCIA DE USUARIO

4.1 Lista de países	**10%**	Lista renderizada eficientemente. Muestra nombre y bandera de cada país. Diseño limpio y consistente. Implementa key props correctamente. Puede incluir búsqueda/filtros.	Lista renderizada correctamente con nombre y bandera. Diseño funcional. Keys implementados.	Lista funcional pero con problemas de rendimiento o diseño descuidado. Puede faltar keys o tener warnings.	Lista no se renderiza correctamente, faltan datos críticos, o hay errores que impiden la visualización.
4.2 Vista de detalles del país	**10%**	Vista de detalles bien estructurada mostrando país, capital y clima. Componentes organizados lógicamente. Transición fluida desde la lista. Diseño coherente.	Vista de detalles funcional mostrando la información requerida. Diseño básico pero presentable.	Vista de detalles muestra información incompleta o el diseño es confuso. Navegación entre vistas problemática.	Vista de detalles ausente, no funcional, o con errores críticos.

5. MANEJO DE ERRORES Y VALIDACIONES

5.1 Validación de datos de APIs	**5%**	Valida que los datos no sean undefined/null antes de renderizar. Usa optional chaining o verificaciones explícitas. Previene crashes por datos faltantes en ambas APIs.	Valida la mayoría de los datos. Puede haber algún edge case sin cubrir pero previene crashes comunes.	Validación parcial. Cubre una API pero no la otra, o hay escenarios donde puede fallar.	Sin validación. La app crashea cuando recibe datos inesperados o faltantes.
5.2 Estados de carga (Loading)	**5%**	Implementa estados de carga claros mientras se obtienen datos. Usa loading spinners o skeletons. La UX durante la espera es profesional.	Muestra indicadores de carga básicos. Puede ser solo texto 'Cargando...' pero informa al usuario.	Estado de carga implementado parcialmente o solo en algunas vistas.	Sin indicadores de carga. La aplicación parece congelada mientras obtiene datos.
5.3 Manejo de errores de red	**5%**	Try/catch implementado en todas las llamadas a API. Mensajes de error claros al usuario. Incluye opción de reintentar o feedback útil.	Try/catch implementado. Muestra mensajes de error al usuario, aunque pueden ser genéricos.	Manejo básico de errores pero principalmente en consola. Feedback limitado al usuario.	Sin manejo de errores. Si las APIs fallan, la app crashea o queda en blanco sin explicación.